

EPSI LYON

Bachelor Développeur en Intelligence Artificielle

Bloc E6.3 – Produire et maintenir une solution I.A — Promotion 2025/2026

RAPPORT TECHNIQUE

MSPR TPRE532

Mise en production d'une solution I.A

ObRail Europe

Observatoire ferroviaire européen — Mobilité durable & Intelligence Artificielle

Équipe projet

Akram • Aymen • Mesanibaumh • Adam

Dépôt GitHub : <https://github.com/AdamSawi/obrail-mspr3>

Juin 2026

1. Introduction et contexte du projet

1.1 ObRail Europe

ObRail Europe est un observatoire indépendant fondé en 2018, spécialisé dans l'analyse des flux ferroviaires européens et la promotion du transport bas-carbone. L'organisation travaille en partenariat avec les institutions européennes, des ONG environnementales comme Transport & Environnement, et les principaux opérateurs ferroviaires — SNCF, DB, Trenitalia, ÖBB Nightjet.

Sa mission repose sur trois axes : collecter et harmoniser les données de desserte ferroviaire à l'échelle européenne, évaluer l'impact environnemental des différents modes de transport longue distance, et produire des études comparatives à destination des décideurs politiques. Dans le cadre du Green Deal européen et du programme TEN-T, ObRail cherche à démontrer que le train constitue une alternative crédible à l'avion sur les trajets intra-européens.

1.2 Contexte de la MSPR TPRE532

Au moment de démarrer cette MSPR, la solution ObRail se trouvait à un stade de prototype non industrialisé : le déploiement restait manuel, les tests n'étaient pas automatisés, et aucun outil de supervision n'assurait la disponibilité du service. L'objectif de cette mission était de franchir l'étape entre un prototype fonctionnel et une application réellement exploitable en production.

Le cahier des charges fixait des exigences précises : stabiliser le backend FastAPI, développer une interface utilisateur accessible, mettre en place une stratégie de tests complète couvrant tests unitaires, d'intégration et end-to-end, automatiser le déploiement via un pipeline CI/CD, et superviser l'application en temps réel.

1.3 Spécifications fonctionnelles

À partir de l'analyse du cahier des charges, l'équipe a identifié les besoins fonctionnels suivants :

- Exposer 142 411 trajets ferroviaires via une API REST paginée et filtrable
- Permettre la consultation par pays (DE, ES, FR, IT), type de traction (electric/diesel), gare de départ, gare d'arrivée et plage de distance
- Afficher des statistiques agrégées : volumes par pays, répartition par type de traction, empreinte CO2 cumulée
- Proposer des prédictions IA : substitution train/avion et estimation CO2 pour un trajet donné
- Exposer un endpoint de santé permettant de vérifier l'état de la base de données et des modèles IA
- Fournir une interface web accessible à un utilisateur non technique
- Assurer la reproductibilité du déploiement en une seule commande
- Superviser le service en temps réel avec des métriques et des logs centralisés

Ces besoins ont guidé l'ensemble des choix de conception et d'implémentation décrits dans ce rapport.

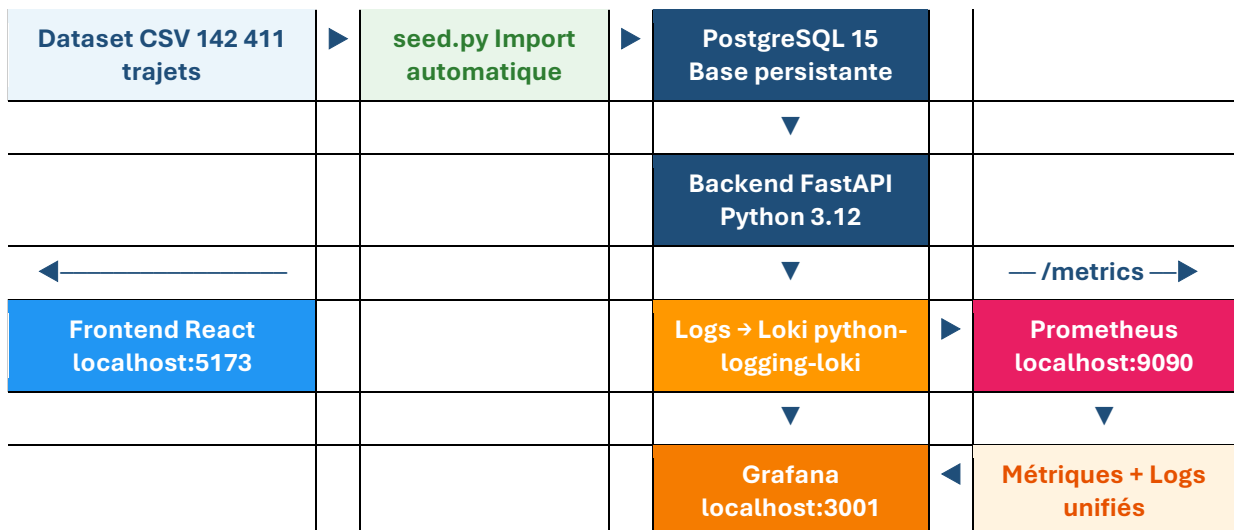
2. Architecture technique

2.1 Vue d'ensemble

La solution repose sur une architecture microservices conteneurisée, orchestrée par Docker Compose. Chaque composant est isolé dans son propre conteneur avec des dépendances strictement définies, ce qui garantit la reproductibilité du déploiement quel que soit l'environnement cible.

Au premier démarrage, le script de seed importe automatiquement le dataset dans PostgreSQL. Le backend FastAPI interroge cette base pour répondre aux requêtes du frontend React. En parallèle, Prometheus collecte les métriques exposées par l'API et Loki centralise les logs applicatifs. Grafana agrège ces deux sources pour offrir une vue unifiée de l'état du système.

2.2 Diagramme d'architecture



Le diagramme ci-dessus représente les flux entre les composants. Le dataset CSV alimente PostgreSQL via le script de seed au démarrage. Le backend FastAPI expose les données au frontend React, envoie ses logs à Loki via python-logging-loki, et expose ses métriques à Prometheus via le endpoint /metrics. Grafana centralise métriques et logs dans un dashboard unique.

2.3 Services et URLs

Service	Port	URL locale	Rôle
PostgreSQL	5432	interne Docker	Base de données persistante
Backend FastAPI	8000	http://localhost:8000	API REST
Swagger	8000	http://localhost:8000/docs	Documentation interactive
Frontend React	5173	http://localhost:5173	Interface utilisateur
Adminer	8081	http://localhost:8081	Administration BDD web

Prometheus	9090	http://localhost:9090	Collecte métriques
Grafana	3001	http://localhost:3001	Dashboards (admin/admin)
Loki	3100	http://localhost:3100	Centralisation des logs

2.4 Choix techniques argumentés

FastAPI

FastAPI a été retenu pour sa performance native (ASGI/asyncio), sa génération automatique de documentation OpenAPI/Swagger, et sa validation de données intégrée via Pydantic. Par rapport à Flask, FastAPI réduit le code boilerplate et produit des APIs mieux documentées dès le départ — un avantage direct pour la démonstration jury via /docs.

PostgreSQL avec SQLAlchemy

Le passage du CSV en lecture directe à PostgreSQL représente la principale évolution architecturale de cette MSPR. PostgreSQL assure la persistance des données entre les redémarrages et permet des requêtes filtrées et paginées directement en SQL, sans charger les 142 411 lignes en mémoire à chaque requête. SQLAlchemy fournit une couche ORM qui facilite les requêtes complexes tout en restant proche du SQL natif.

React avec Vite

Vite offre un démarrage quasi instantané en développement et des builds de production optimisés. L'interface a été conçue pour un public non technique — institutionnel, ONG ou opérateur ferroviaire — avec une attention portée à la lisibilité des données et à la fluidité de navigation.

Prometheus + Grafana + Loki

Cette stack constitue le standard de l'industrie pour la supervision d'applications conteneurisées. Prometheus collecte les métriques via /metrics, Grafana les visualise dans un dashboard provisionné automatiquement, et Loki centralise les logs envoyés directement depuis FastAPI. L'ensemble fonctionne sans configuration manuelle après le démarrage.

3. Base de données et gestion des données

3.1 Modèle de données

La table trips contient 24 colonnes couvrant l'ensemble des attributs d'un trajet ferroviaire : identifiants de ligne et de service, gares d'origine et de destination avec coordonnées GPS, horaires de départ et d'arrivée, distance, durée, nombre d'arrêts, type de traction et empreinte CO2.

La clé primaire est un identifiant auto-incrémenté. trip_id est indexé avec une contrainte d'unicité pour garantir l'idempotence du seed. Les colonnes country et type_train sont également indexées car elles constituent les filtres les plus fréquemment utilisés. Le champ route_long_name est nullable dans environ 44% des cas, ce qui reflète la réalité des données open data ferroviaires européennes.

3.2 Script de seed

Le script seed.py est idempotent : il peut être lancé plusieurs fois sans créer de doublons. Au démarrage, il charge les trip_id existants en base puis parcourt le CSV en ignorant les lignes déjà présentes. L'insertion se fait par batches de 1 000 lignes via bulk_insert_mappings, ce qui réduit le nombre de transactions et accélère l'import.

Plusieurs corrections de qualité sont appliquées : filtrage des trajets avec une durée nulle (9 lignes aberrantes), normalisation des types mixtes, et détection des valeurs CO2 anormalement élevées avec un warning loggé.

- 142 420 lignes lues dans le CSV
- 9 lignes filtrées (duration_minutes == 0)
- 142 411 lignes insérées en base

3.3 Pagination SQL

L'approche initiale chargeait l'intégralité des 142 411 trajets en mémoire via pandas puis appliquait les filtres côté Python — rendant chaque requête lente. La solution retenue délègue filtrage et pagination à PostgreSQL. La fonction _build_trips_query() construit dynamiquement une requête SQLAlchemy avec les filtres actifs, et list_trajets exécute un COUNT puis un SELECT avec LIMIT/OFFSET pour ne charger que les lignes de la page courante. Le temps de réponse est passé de plusieurs secondes à moins de 100ms.

3.4 Visualisation avec Adminer

Adminer est intégré à la stack Docker et accessible sur le port 8081. Il permet d'inspecter la base PostgreSQL depuis un navigateur sans outil externe. La preuve de bon fonctionnement est également vérifiable via le script scripts/preuve_postgresql.sh qui affiche le schéma de la table trips et le nombre de lignes.

```
bash scripts/preuve_postgresql.sh
```

4. Backend FastAPI

4.1 Endpoints

Méthode	Route	Description
GET	/health	État de l'API, de la BDD et des modèles
GET	/trajets	Liste paginée avec filtres (pays, type, gares, distance)
GET	/trajets/{id}	Détail d'un trajet par son identifiant
GET	/stats/volumes	Agrégats CO2 et volumes par pays et type de traction
GET	/metrics	Métriques Prometheus au format texte
POST	/predict/substitution	Prédiction substitution train/avion (XGBoost)
POST	/predict/co2	Régression CO2 pour un trajet donné (XGBoost)
GET	/docs	Documentation Swagger auto-générée

4.2 Gestion des erreurs

Toutes les erreurs sont renvoyées avec une structure JSON homogène via `_error_payload`, contenant le code d'erreur, un message lisible, le code HTTP, un timestamp et le chemin de la requête. Les handlers couvrent trois niveaux : erreurs de validation Pydantic (422), erreurs HTTP standard, et exceptions non gérées (500) loggées intégralement côté serveur avant d'être renvoyées de façon simplifiée au client.

4.3 Modèles IA

Quatre artefacts joblib constituent le module d'intelligence artificielle. Le modèle de classification prédit si un trajet ferroviaire peut se substituer à l'avion avec une probabilité associée. Le modèle de régression estime la consommation CO2. Un point critique a été corrigé lors de l'intégration : le `StandardScaler` devait être appliqué avant toute prédiction de substitution car le modèle XGBoost avait été entraîné sur des features standardisées. Sans ce correctif, les prédictions étaient biaisées de façon silencieuse.

4.4 Documentation Swagger

FastAPI génère automatiquement une interface Swagger complète sur `/docs`, listant tous les endpoints avec leurs paramètres, schémas de réponse et exemples. Cela permet de tester interactivement les routes de prédiction lors de la démonstration jury sans outil externe.

ObRail Europe — API de prédiction ferroviaire

2.0.0 OAS 3.1

/openapi.json

API REST du projet ObRail Europe. Deux enjeux exposés :

- **Substitution avion/train** : classification binaire — la liaison est-elle candidate à remplacer un vol aérien ?
- **Emissions CO2 futures** : régression — estimation des émissions kgCO2 selon un scénario de développement du réseau.

Contact ObRail Europe

Général		^
GET	/ Root	▼
GET	/health Health	▼
Données		^
GET	/trajets List Trajets	▼
GET	/trajets/{trajet_id} Get Trajet	▼
GET	/stats/volumes Stats Volumes	▼
Classification		^
POST	/predict Predict Compat	▼
POST	/predict/substitution Predict Substitution	▼
Régression CO2		^
POST	/predict/co2 Predict Co2	▼

5. Frontend React

5.1 Interface utilisateur

L'interface est une single page application React/Vite ciblant un utilisateur non technique mais exigeant. La page principale affiche immédiatement quatre indicateurs globaux récupérés dynamiquement depuis l'API : 142 411 trajets disponibles, 4 pays couverts (DE, ES, FR, IT), 2 191 routes distinctes, et 35 770 606 kgCO2 cumulés.

5.2 Fonctionnalités

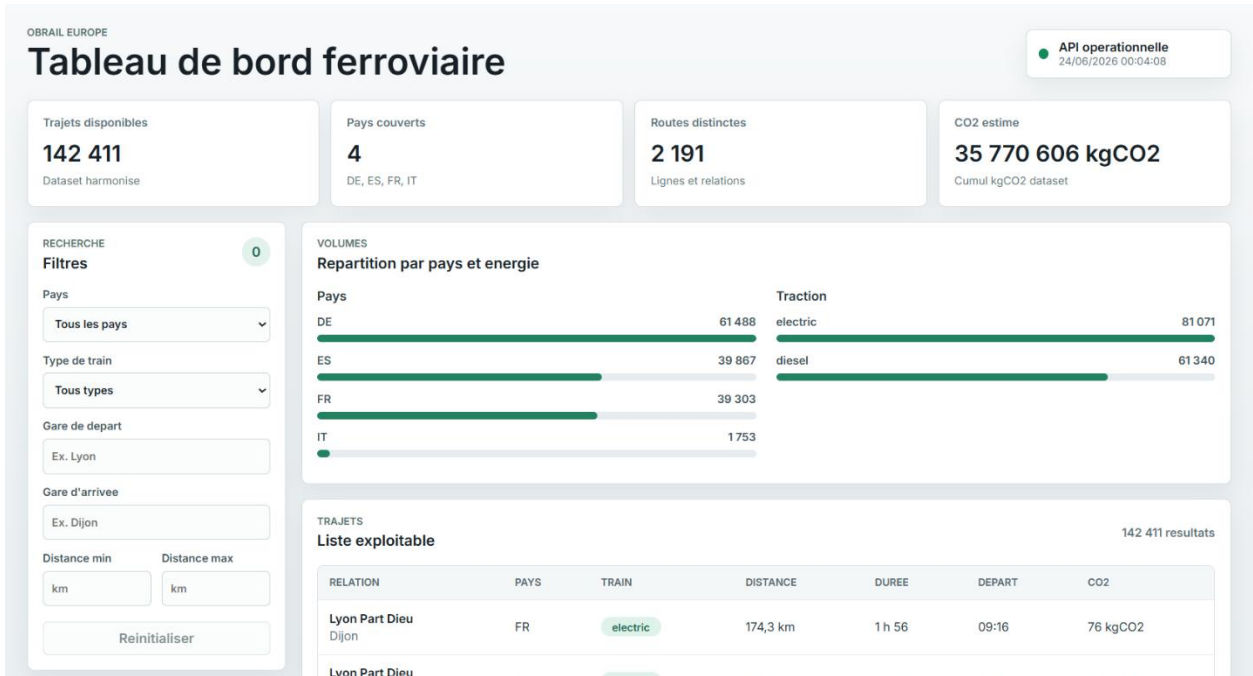
- Filtrage des trajets par pays, type de traction, gare de départ, gare d'arrivée et plage de distance
- Pagination fluide sur les 5 697 pages de résultats
- Visualisation des volumes par pays et type de traction avec barres proportionnelles
- Indicateur de statut de l'API en temps réel (API opérationnelle / indisponible)
- Affichage de l'empreinte CO2 par trajet en kgCO2

5.3 Accessibilité RGAA

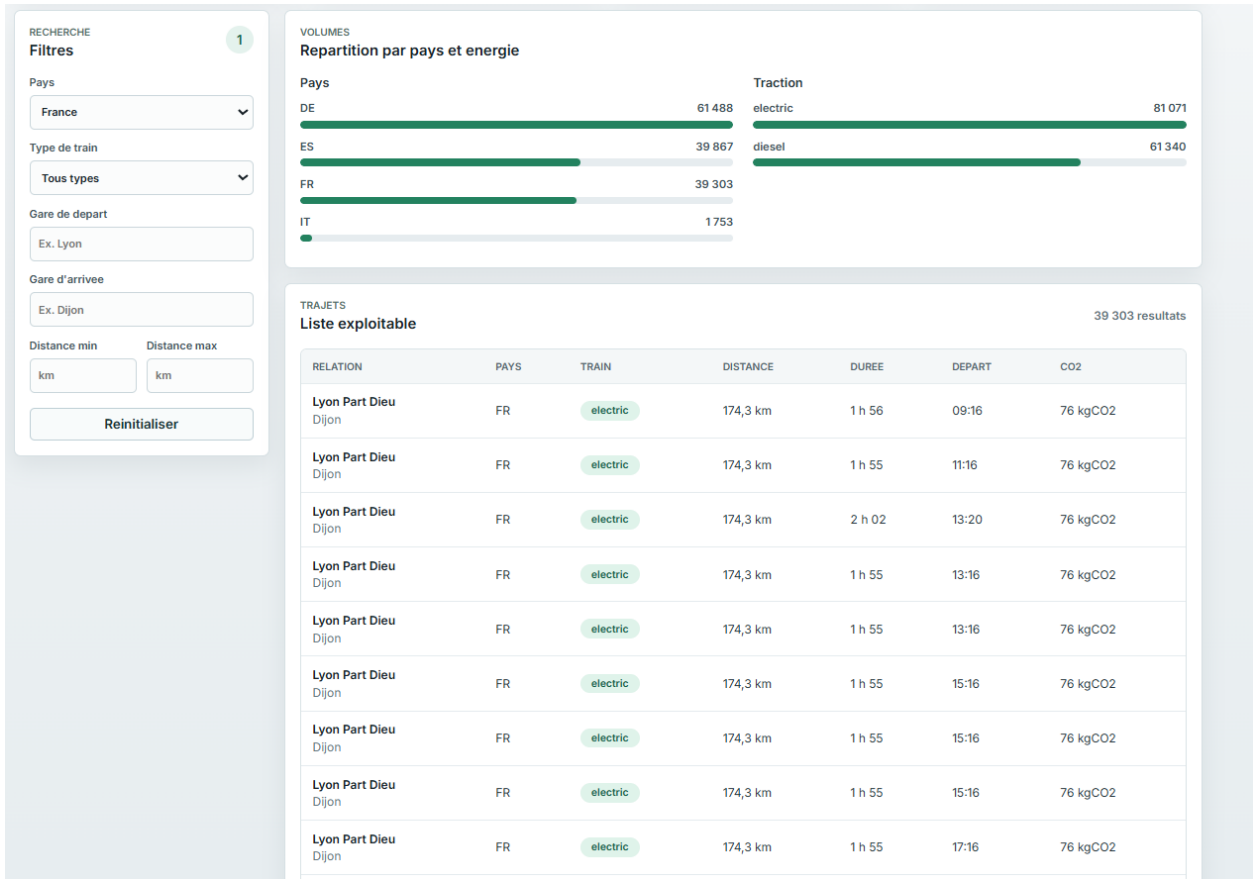
L'interface frontend a été développée en intégrant les recommandations du Référentiel Général d'Amélioration de l'Accessibilité dès la conception, sans audit formel a posteriori.

Mesures mises en œuvre et vérifiées dans le code source :

- Langue déclarée : l'attribut lang="fr" est présent sur la balise html, permettant aux lecteurs d'écran d'adapter la prononciation
- Lien d'évitement : un lien "Aller aux trajets" permet aux utilisateurs naviguant au clavier de sauter directement au contenu principal
- Sémantique HTML5 : utilisation des balises <main>, <header>, <aside>, <nav>, <article> et <section> avec des rôles explicites
- Labels sur tous les champs : chaque input et select dispose d'un aria-label descriptif (filtrer par pays, filtrer par gare de départ, distance minimale en kilomètres...)
- Mises à jour dynamiques annoncées : aria-live="polite" sur le compteur de résultats et la pagination, aria-busy sur les sections en chargement
- Messages d'erreur accessibles : role="alert" sur toutes les sections d'erreur pour notification immédiate aux lecteurs d'écran
- Éléments décoratifs masqués : aria-hidden="true" sur le point de statut visuel du badge API
- Tableau de données structuré : attribut scope="col" sur tous les en-têtes de colonne
- Boutons désactivés explicitement : les boutons de pagination sont en état disabled quand l'action n'est pas disponible



Filtre appliqué : France



6. Conteneurisation et orchestration Docker

6.1 Organisation des services

La stack est définie dans `docker/docker-compose.yml` avec huit services. L'ordre de démarrage est géré par des dépendances `condition: service_healthy` garantissant que chaque service attend que ses prérequis soient opérationnels. PostgreSQL démarre en premier avec un healthcheck `pg_isready`, le backend attend la base, le frontend attend le backend.

6.2 Script d'initialisation

Le fichier `scripts/entrypoint.sh` orchestre le démarrage du backend en trois phases : attente active de PostgreSQL via `pg_isready`, lancement de `seed.py` pour importer les données si la base est vide, puis démarrage d'uvicorn. L'idempotence du `seed` garantit des relances sans doublons.

6.3 Persistance

Trois volumes nommés assurent la persistance entre redémarrages : `postgres-data` pour les données PostgreSQL, `grafana-data` pour les dashboards, `loki-data` pour les logs. Données et modèles sont montés en lecture seule dans le conteneur backend.

6.4 Lancement en une seule commande

```
docker compose -f docker/docker-compose.yml up --build
```

Cette commande construit les images, démarre tous les services dans l'ordre, seed la base automatiquement, et rend l'application accessible en deux à trois minutes.

7. Pipeline CI/CD — GitHub Actions

7.1 Déclenchement et structure

Le pipeline `.github/workflows/ci.yml` se déclenche sur chaque push et pull request. Il comprend quatre jobs s'exécutant dans un ordre logique : backend et frontend en parallèle, puis docker après les deux, puis e2e après docker.

7.2 Jobs

Job	Étapes principales
Backend	Démarrage PostgreSQL 15 en service CI, install dépendances Python, vérification artefacts, compilation, seed, Pytest (12 tests)
Frontend	Install Node.js 22, tests unitaires JavaScript, build Vite
Docker	Validation syntaxe docker-compose.yml, build images backend et frontend
E2E	Launch stack Docker Compose, install Playwright + Chromium, Playwright tests (4 tests)

7.3 Gestion des variables

Les credentials PostgreSQL sont définis dans le fichier CI via la clé env du service GitHub Actions. En production réelle, ces valeurs seraient stockées dans les secrets GitHub et injectées via la syntaxe `${{ secrets.POSTGRES_PASSWORD }}`.

7.4 Coordination agile et MLOps

Le projet a été conduit selon une approche agile itérative, organisée autour d'un board Trello structuré en colonnes Backlog, À faire, En cours et Terminé. Les tâches ont été découpées par composant technique — Backend, Frontend, Docker, Tests, CI/CD, Monitoring, Documentation — permettant à chaque membre de l'équipe de travailler en parallèle sur son périmètre sans bloquer les autres.

Le travail a progressé par itérations successives : cadrage et définition de l'architecture cible en premier, puis stabilisation du backend et des routes API, mise en place de PostgreSQL et du script d'import, développement du frontend, conteneurisation Docker Compose, et enfin intégration des tests automatisés et du monitoring. Chaque composant livré a été vérifié et intégré avant de passer au suivant, ce qui a permis de détecter les problèmes tôt — notamment la correction du StandardScaler sur les prédictions de substitution et l'optimisation de la pagination SQL.

Le versioning a été géré via Git avec des branches de fonctionnalités mergées sur la branche principale après validation. Cette organisation s'inscrit directement dans une démarche MLOps : chaque push sur le dépôt déclenche automatiquement le pipeline GitHub Actions — installation des dépendances, seed PostgreSQL, tests Pytest, build Docker, tests Playwright — garantissant que la qualité de la solution est vérifiée en continu sans intervention manuelle.

8. Stratégie de tests

8.1 Tests backend — Pytest

Les tests couvrent l'ensemble des routes via une base SQLite en mémoire — aucune dépendance externe nécessaire, tests rapides et isolés. La fixture `conftest.py` crée la base, insère dix trajets représentatifs (quatre pays, deux types de traction) et fournit un client FastAPI de test réinitialisé à chaque test.

Test	Résultat	Description
<code>test_root_documents_operational_routes</code>	PASSED	Routes racine et Swagger accessibles
<code>test_list_trajets_default_pagination</code>	PASSED	Pagination par défaut (25 résultats)
<code>test_list_trajets_filters_by_country_and_train_type</code>	PASSED	Filtres pays et type de traction
<code>test_list_trajets_filters_by_origin_and_destination_text</code>	PASSED	Recherche par nom de gare
<code>test_list_trajets_rejects_inconsistent_distance_bounds</code>	PASSED	Validation des bornes de distance
<code>test_validation_errors_use_homogeneous_payload</code>	PASSED	Format d'erreur homogène
<code>test_get_trajet_by_stable_id_matches_list_item</code>	PASSED	Cohérence liste / détail
<code>test_get_trajet_returns_404_for_unknown_id</code>	PASSED	Gestion 404
<code>test_stats_volumes_exposes_global_and_grouped_counts</code>	PASSED	Agrégats volumes
<code>test_health_includes_dataset_models_and_timestamp</code>	PASSED	Health check complet
<code>test_cors_allows_local_frontend_origin</code>	PASSED	Configuration CORS
<code>test_metrics_exposes_prometheus_text</code>	PASSED	Exposition métriques Prometheus

Résultat : 12 passed en 0.50s.

```
-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== 12 passed, 18 warnings in 0.50s =====
```

```
collected 12 items

backend/tests/test_placeholder.py::test_root_documents_operational_routes PASSED [ 8%]
backend/tests/test_trajets.py::test_list_trajets_default_pagination PASSED [ 16%]
backend/tests/test_trajets.py::test_list_trajets_filters_by_country_and_train_type PASSED [ 25%]
backend/tests/test_trajets.py::test_list_trajets_filters_by_origin_and_destination_text PASSED [ 33%]
backend/tests/test_trajets.py::test_list_trajets_rejects_inconsistent_distance_bounds PASSED [ 41%]
backend/tests/test_trajets.py::test_validation_errors_use_homogeneous_payload PASSED [ 50%]
backend/tests/test_trajets.py::test_get_trajet_by_stable_id_matches_list_item PASSED [ 58%]
backend/tests/test_trajets.py::test_get_trajet_returns_404_for_unknown_id PASSED [ 66%]
backend/tests/test_trajets.py::test_stats_volumes_exposes_global_and_grouped_counts PASSED [ 75%]
backend/tests/test_trajets.py::test_health_includes_dataset_models_and_timestamp PASSED [ 83%]
backend/tests/test_trajets.py::test_cors_allows_local_frontend_origin PASSED [ 91%]
backend/tests/test_trajets.py::test_metrics_exposes_prometheus_text PASSED [100%]
```

8.2 Tests E2E — Playwright

Playwright pilote un navigateur Chromium en mode headless et vérifie le comportement réel de l'interface. Ces tests valident l'intégration complète frontend → API → base de données.

Test	Résultat	Description
test_page_charge_et_affiche_les_donnees	PASSED	Page charge et affiche 142 411 trajets
test_filtre_par_pays_fonctionne	PASSED	Filtre par pays FR fonctionnel
test_navigation_repond_sans_erreur_serveur	PASSED	Aucune erreur 500 sur la navigation
test_api_health_visible_dans_le_frontend	PASSED	Indicateur API opérationnelle visible

Résultat : 4 passed en 29.95s.

```
collected 4 items

frontend/tests/test_e2e.py::test_page_charge_et_affiche_les_donnees PASSED [ 25%]
frontend/tests/test_e2e.py::test_filtre_par_pays_fonctionne PASSED [ 50%]
frontend/tests/test_e2e.py::test_navigation_repond_sans_erreur_serveur PASSED [ 75%]
frontend/tests/test_e2e.py::test_api_health_visible_dans_le_frontend PASSED [100%]

===== 4 passed in 29.95s =====
```

8.3 Commandes

```
python -m pytest backend/tests/ -v      # 12 passed
python -m pytest frontend/tests/test_e2e.py -v  # 4 passed
```

9. Supervision et observabilité

9.1 Architecture de monitoring

La supervision couvre deux dimensions complémentaires : les métriques quantitatives (Prometheus + Grafana) et les logs qualitatifs (Loki + Grafana). Les deux sources sont unifiées dans le même outil de visualisation.

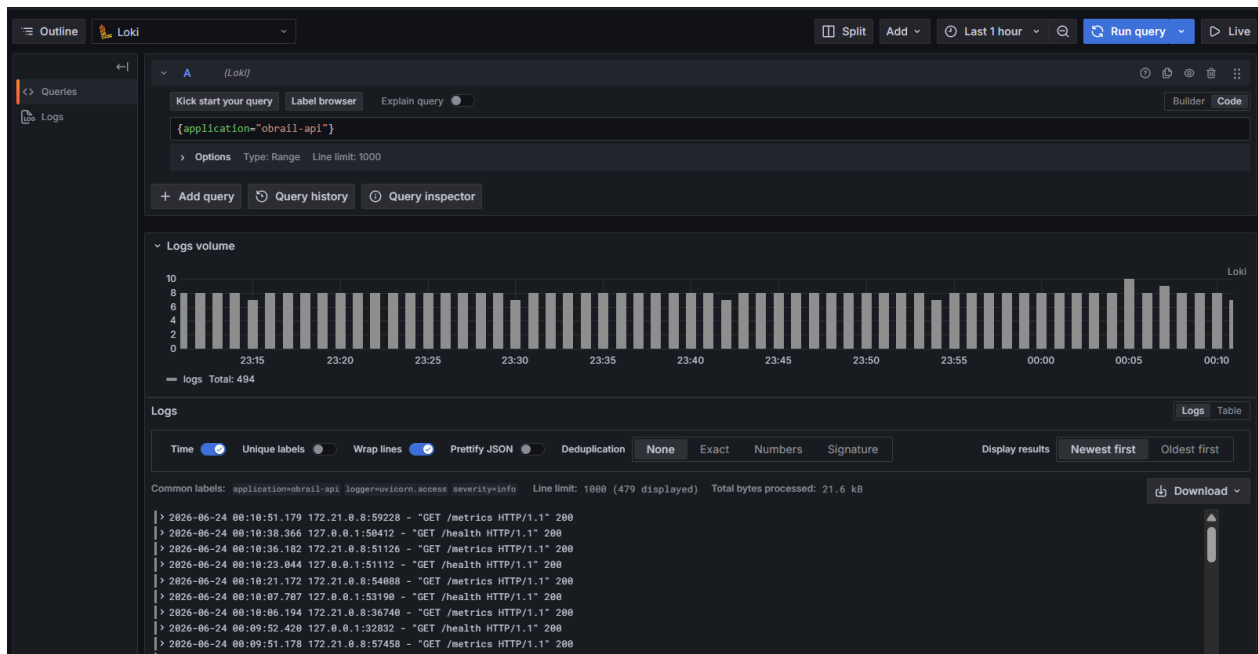
9.2 Métriques Prometheus

Le backend expose /metrics via prometheus-fastapi-instrumentator. Prometheus scrape ce endpoint toutes les 15 secondes. Le dashboard Grafana affiche cinq panneaux : statut Dataset (Up/Down en temps réel), volume de données (142 K), trafic en req/s, répartition par route, et latence P95.

- Disponibilité : Dataset disponible / Up ou Down
- Volume : 142 411 trajets en base
- Trafic : nombre de requêtes par seconde par route
- Latence : percentile P95 du temps de réponse

9.3 Logs applicatifs avec Loki

Les logs FastAPI sont envoyés directement à Loki via python-logging-loki avec le tag application=obrail-api. Chaque log contient la méthode HTTP, le chemin, le code de statut et l'IP source. Lors des tests de validation, 185 logs ont été collectés et affichés dans Grafana Explore avec la requête {application="obrail-api"}.



9.4 Dashboard Grafana

Le dashboard ObRail API est provisionné automatiquement via monitoring/grafana/dashboards/obrail-api.json — aucune configuration manuelle après redémarrage. Grafana est accessible sur le port 3001 avec les identifiants admin/admin.



10. Sécurité, RGPD et accessibilité

10.1 Conformité RGPD

Le dataset ne contient aucune donnée à caractère personnel — uniquement des liaisons entre gares avec distances, durées et empreintes CO2. Bonnes pratiques appliquées :

- Aucun cookie de tracking dans le frontend
- Logs limités aux métadonnées de requête sans corps de requête
- Variables d'environnement pour tous les credentials, jamais codés en dur
- Dataset et modèles montés en lecture seule dans les conteneurs
- CORS limité aux origines locales connues
- Droits GitHub Actions limités à la lecture du dépôt

10.2 Sécurité applicative

Validation des entrées assurée par Pydantic sur tous les endpoints. Les erreurs 500 sont loggées intégralement côté serveur mais renvoyées de façon simplifiée au client pour ne pas exposer de traces internes. Points à renforcer pour une production réelle : authentification JWT, HTTPS, rotation des secrets Grafana.

10.3 Accessibilité RGAA

Les mesures d'accessibilité mises en œuvre dans le frontend sont détaillées en section 5.3. L'interface respecte les recommandations RGAA de base, vérifiées directement dans le code source lors du développement.

11. Procédures d'installation et de vérification jury

11.1 Prérequis

- Docker Desktop installé et en cours d'exécution
- Git pour cloner le dépôt
- Ports disponibles : 5432, 8000, 5173, 3001, 9090, 8081, 3100

11.2 Lancement complet

```
git clone https://github.com/AdamSawi/obrail-mspr3.git
cd obrail-mspr3
docker compose -f docker/docker-compose.yml up --build
```

Première fois : prévoir 3 à 5 minutes. Les relances suivantes démarrent en moins d'une minute.

11.3 Vérification de l'API

```
curl http://localhost:8000/health
# Attendu : status ok, rows 142411, modèles ok
curl http://localhost:8000/trajets?page_size=1
# Attendu : total 142411
bash scripts/preuve_postgresql.sh
# Attendu : table trips, 142411 lignes
```

11.4 Interfaces disponibles

Frontend	http://localhost:5173
API Swagger	http://localhost:8000/docs
Grafana	http://localhost:3001 — admin / admin
Prometheus	http://localhost:9090
Adminer (BDD)	http://localhost:8081 — PostgreSQL / db / obrail / obrail

11.5 Tests

```
pip install psycpg2-binary python-logging-loki playwright
playwright install chromium
python -m pytest backend/tests/ -v # 12 passed
python -m pytest frontend/tests/test_e2e.py -v # 4 passed
```


12. Stratégie de maintenance et rollback

12.1 Mise à jour

```
git pull origin main
docker compose -f docker/docker-compose.yml up --build -d
```

12.2 Rollback

En cas de problème après une mise à jour, revenir au commit précédent puis relancer la stack. La persistance des volumes garantit que les données PostgreSQL ne sont pas perdues.

```
git log --oneline -5
git checkout <hash-du-commit-stable>
docker compose -f docker/docker-compose.yml up --build -d
```

12.3 Sauvegarde et restauration

```
# Sauvegarde
docker exec obrail-db pg_dump -U obrail obrail > backup.sql
# Restauration
docker exec -i obrail-db psql -U obrail obrail < backup.sql
```

12.4 Arrêt

```
docker compose -f docker/docker-compose.yml down          # données
conservées
docker compose -f docker/docker-compose.yml down -v       # données
supprimées
```

13. Synthèse

13.1 Récapitulatif des livrables

- Application complète exécutable via docker compose -f docker/docker-compose.yml up --build
- Dockerfiles backend et frontend, docker-compose.yml avec 8 services orchestrés
- Pipeline CI/CD GitHub Actions : 4 jobs (backend, frontend, docker, e2e)
- 12 tests Pytest backend + 4 tests Playwright E2E frontend, tous intégrés en CI
- Dashboard Grafana avec métriques Prometheus et logs Loki opérationnels
- Rapport technique (ce document) et README sur le dépôt GitHub

13.2 Points forts

- Pagination SQL native : les requêtes /trajets ne chargent que les lignes de la page courante, temps de réponse inférieur à 100ms
- Seed idempotent : peut être lancé plusieurs fois sans doublons, relances sécurisées
- Monitoring complet : métriques + logs + dashboard dans un seul outil, sans configuration manuelle
- CI/CD robuste : 4 jobs couvrant backend, frontend, Docker et E2E, déclenchés sur chaque push
- Correction du StandardScaler : les prédictions de substitution avion étaient biaisées sans ce correctif identifié lors de l'intégration

13.3 Limites et axes d'amélioration

- Compatibilité sklearn : les modèles ont été entraînés avec sklearn 1.8.0, l'environnement local utilise la 1.6.1 — une standardisation de l'environnement d'entraînement éliminerait les warnings
- Authentification : l'API est ouverte sans authentification — un système JWT serait nécessaire pour une production réelle
- HTTPS : non configuré dans cette version, prérequis pour tout déploiement public

Annexes

A. Structure du dépôt

```
obrail-mspr3/
├── backend/
│   ├── app/
│   │   ├── main.py          # API FastAPI
│   │   ├── database.py      # Connexion SQLAlchemy
│   │   └── models.py        # Modèle Trip (24 colonnes)
│   ├── tests/
│   │   ├── conftest.py      # Fixtures SQLite en mémoire
│   │   ├── test_trajets.py  # 11 tests API
│   │   └── test_placeholder.py
│   ├── seed.py              # Import CSV → PostgreSQL
│   ├── requirements.txt
│   └── Dockerfile
├── frontend/
│   ├── src/
│   │   ├── main.jsx
│   │   └── services/api.js
│   ├── tests/test_e2e.py    # 4 tests Playwright
│   └── Dockerfile
├── data/eu_trips_v2.csv      # 142 420 trajets
├── models/                  # classification, regression, encoders,
scaler
├── docker/docker-compose.yml
├── monitoring/              # Prometheus, Grafana, Loki, Promtail
├── scripts/entrypoint.sh
├── .github/workflows/ci.yml
└── RAPPORT_TECHNIQUE.md
```

B. Variables d'environnement

DATABASE_URL	postgresql://obrail:obrail@db:5432/obrail
CSV_PATH	/app/data/eu_trips_v2.csv
DB_HOST	db (Docker) / localhost (local)
DB_USER	obrail
OBRAIL_CORS_ORIGINS	http://localhost:5173,http://127.0.0.1:5173
VITE_API_BASE_URL	http://localhost:8000
GRAFANA_ADMIN_USER	admin
GRAFANA_ADMIN_PASSWORD	admin

C. Compétences RNCP36581 — Bloc E6.3

Compétence évaluée	Démonstration dans le projet
Analyser le besoin	Spécifications fonctionnelles rédigées, cahier des charges ObRail traité
Concevoir l'architecture	Architecture microservices documentée avec diagramme de flux
Coordonner en MLOps	Pipeline CI/CD + versioning des modèles joblib
Développer les composants	FastAPI + React + PostgreSQL + SQLAlchemy
Automatiser les tests	12 Pytest + 4 Playwright intégrés en CI sur chaque push
Livraison continue	GitHub Actions 4 jobs, déclenchement automatique push/PR
Surveiller l'application	Prometheus + Grafana dashboard + Loki 185 logs collectés
Résoudre les incidents	Correction StandardScaler + pagination SQL + seed idempotent